

Bento Grid 響應式排版網站 - 實戰教學手冊

這份手冊將帶你一步步從零開始，使用 **React (Vite)** 搭配 **Tailwind CSS**，建立一個現代化、動態且極具質感的 Bento Grid（便當盒網格）網站。

步驟一：建立專案環境

首先，我們需要建立一個新的 React 專案，並安裝必要的依賴套件。

1. 打開你的終端機 (Terminal) 或命令提示字元。
2. 進入你想放置專案的資料夾，例如我們現在的桌面 Bento-grid 目錄：

```
cd c:\Users\1411\Desktop\Bento-grid
```

3. 建立一個名為 bento-grid-app 的新專案：

```
npm create vite@latest bento-grid-app -- --template react
```

4. 進入專案資料夾並安裝基本套件：

```
cd bento-grid-app
npm install
```

5. 安裝 Tailwind CSS 與其相關套件：

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

6. （選用）安裝動畫套件 framer-motion，為版面增添未來感：

```
npm install framer-motion
```

步驟二：設定 Tailwind CSS

我們需要告訴 Tailwind 去哪裡尋找我們寫的 class。

1. 打開專案中的 tailwind.config.js，將內容修改為：

```
/** @type {import('tailwindcss').Config} */
export default {
  content: [
    "./index.html",
    "./src/**/*.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

2. 打開 src/index.css，清空原本的內容，並加入以下 Tailwind 基本指令與自訂背景：

```
@tailwind base;
@tailwind components;
@tailwind utilities;

body {
  background-color: #0f172a; /* 深色沉穩背景 */
  color: #f8fafc;
```

步驟三：設計 Bento Grid 網格佈局

Bento Grid 的核心在於 CSS Grid。我們將在主程式中建立網格容器。

1. 打開 src/App.jsx，清空原本的代碼，替換成以下的架構：

```
import React from 'react';

function App() {
  return (
    <div className="min-h-screen bg-slate-900 p-8 flex items-center justify-center">
      /* 這是 Bento Grid 的主容器 */
      <div className="grid grid-cols-1 md:grid-cols-4 gap-4 max-w-6xl w-full auto-rows-[200px]">

        /* 卡片 1 - 佔據較大版面 */
        <div className="col-span-1 md:col-span-2 md:row-span-2 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700">
          <h2 className="text-3xl font-bold mb-4">主打視覺區</h2>
          <p className="text-slate-400">這裡是內容最豐富的區塊，適合放主要視覺、核心理念或大張圖表。</p>
        </div>

        /* 卡片 2 - 寬扁型 */
        <div className="col-span-1 md:col-span-2 bg-gradient-to-br from-indigo-500 to-purple-600 rounded-3xl p-6 shadow-lg flex items-end">
          <h2 className="text-2xl font-semibold text-white">高光色彩區</h2>
        </div>

        /* 卡片 3 - 方型 */
        <div className="col-span-1 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700 flex flex-col justify-center items-center">
          <h3 className="text-xl font-medium">數據 1</h3>
          <p className="text-3xl text-indigo-400 font-bold mt-2">+80%</p>
        </div>

        /* 卡片 4 - 方型 */
        <div className="col-span-1 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700 flex flex-col justify-center items-center">
          <h3 className="text-xl font-medium">數據 2</h3>
          <p className="text-3xl text-purple-400 font-bold mt-2">>120k</p>
        </div>

        /* 卡片 5 - 橫跨三格 */
        <div className="col-span-1 md:col-span-3 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700">
          <h3 className="text-xl font-medium mb-2">更多資訊</h3>
          <p className="text-slate-400">橫長型的區塊適合拿來做步驟說明、文章列表或是橫向捲動的圖集。</p>
        </div>

        /* 卡片 6 - 小正方型 */
        <div className="col-span-1 bg-gradient-to-tr from-pink-500 to-orange-400 rounded-3xl p-6 shadow-lg">
          <h3 className="text-xl font-medium text-white">加入我們</h3>
        </div>

      </div>
    </div>
  );
}

export default App;
```

🔗 網格原理解析：

- grid-cols-4: 讓畫面總共分成 4 個欄位。
- auto-rows-[200px]: 每一列的預設高度固定為 200px。
- col-span-x: 設定該卡片要橫向跨越多個欄位（例如 2 格）。
- row-span-x: 設定該卡片要垂直跨越多個列（例如 2 格）。

步驟四：加入微動畫（質感升級）

為了讓網站有「高級感」，我們可以利用剛剛安裝的 framer-motion。

1. 在 App.jsx 最上方引入 motion：

```
import { motion } from 'framer-motion';
```

2. 將你的 <div className="col-span-1..."> 卡片替換成 <motion.div>，並加上 whileHover 與 transition 屬性，例如：

```
<motion.div
  whileHover={{ scale: 1.02 }}
  transition={{ type: "spring", stiffness: 300 }}
  className="col-span-1 md:col-span-2 md:row-span-2 bg-slate-800 rounded-3xl p-6 shadow-lg border border-slate-700"
>
  /* 內容 */
</motion.div>
```

💡 提示：你可以把每個卡片都加上 motion.div，這會讓滑鼠移過去時，卡片有輕微回彈放大的高級手感喔！

步驟五：啟動並預覽成果

在終端機中確保你在 bento-grid-app 資料夾內，執行以下指令啟動開發伺服器：

```
npm run dev
```

打開瀏覽器前往終端機顯示的網址（通常是 http://localhost:5173），你就會看到極具設計感的 Bento Grid 排版呈現在你眼前了！🚀

🔗 下一步挑戰？

1. 自訂排版：試著修改各個卡片的 col-span 和 row-span 的數字，玩出屬於你自己的版面。
2. 加入圖標：安裝 lucide-react，在卡片中放入漂亮的 Icons。
3. 響應式調整：在手機版時（md: 以外的狀態），觀察卡片是如何自動變成單欄排列的，並調整間距。

🌟 額外進階：引進 TradingView 專業看盤圖表與畫圖工具

如果你想讓這個網頁具備「專業看盤」的功能（包含畫線、拉區間、甚至看基本面與法人），完全不需要自己接 API 刻到頭破血流，我們可以直接使用現成的 TradingView 核心！

1. 安裝套件：在終端機輸入以下指令安裝

```
npm install react-ts-tradingview-widgets
```

2. 在你的頁面中使用圖表：你可以建立一個像是 AnalysisView.jsx 的新元件，把最夯的進階圖表放進去：

```
import React from 'react';
import { AdvancedRealTimeChart, CompanyProfile, SymbolInfo } from "react-ts-tradingview-widgets";

export default function AnalysisView() {
  return (
    <div className="flex flex-col gap-6 p-8">
      {/* 上方放置大圖表 (內建畫線工具、K線上萬種指標) */}
      <div className="h-[600px] rounded-2xl overflow-hidden border border-slate-700 shadow-xl">
        <AdvancedRealTimeChart symbol="NASDAQ:AAPL" theme="dark" autosize />
      </div>

      {/* 下方放置財報與基本分析 */}
      <div className="grid grid-cols-2 gap-6 h-[400px]">
        <div className="rounded-2xl overflow-hidden border border-slate-700">
          <CompanyProfile symbol="NASDAQ:AAPL" colorTheme="dark" width="100%" height="100%" />
        </div>
        <div className="rounded-2xl overflow-hidden border border-slate-700">
          <SymbolInfo symbol="NASDAQ:AAPL" colorTheme="dark" autosize />
        </div>
      </div>
    </div>
  );
}
```

這招能讓你的專案瞬間提升到「專業 FinTech 網站」的等級喔！

✎ 最終進階：加入全球測向儀與即時新聞牆

如果你希望你的網站不只有圖表，還能像 Bloomberg (彭博社) 一樣不斷跳出最新的華爾街即時新聞，以及判斷目前市場情緒的「技術分析儀表板」，你同樣可以使用 react-ts-tradingview-widgets 中的強大組件！

1. 引入套件中的 Timeline 與 TechnicalAnalysis：

```
import React from 'react';
import { Timeline, TechnicalAnalysis } from "react-ts-tradingview-widgets";

export default function MarketNewsView() {
  return (
    <div className="flex flex-col md:flex-row gap-6 p-8 h-screen bg-slate-900 border border-slate-700">

      {/* 區塊 1：動態市場買賣情緒指針 */}
      <div className="flex-1 rounded-2xl overflow-hidden bg-slate-800 p-4 relative">
        <h3 className="text-white font-bold mb-4 text-xl">S&P 500 市場情緒</h3>
        {/* 這個指針會根據真實技術指標自動告訴你現在是 Strong Buy 還是 Strong Sell */}
        <TechnicalAnalysis symbol="SPY" colorTheme="dark" width="100%" height={400} />
      </div>

      {/* 區塊 2：不斷更新的全球即時新聞牆 */}
      <div className="flex-1 rounded-2xl overflow-hidden bg-slate-800 p-4 h-[600px]">
        <h3 className="text-white font-bold mb-4 text-xl">即時滾動新聞</h3>
        <Timeline colorTheme="dark" feedMode="market" market="stock" height="100%" width="100%" />
      </div>
    </div>
  );
}
```

只要貼上這段程式碼，你的網站就能擁有會發亮的動態儀表板以及無窮無盡的即時財經新聞，讓作品的完整度破表！

💧 大師級密技：零後端！突破 5MB 限制的超流暢頭貼上傳器

在無伺服器 (Serverless) 的純前端架構中，我們通常會使用 localStorage 來永久記憶使用者的資料。但是，瀏覽器的本機空間通常有 **5MB** 的大小限制！如果使用者上傳了一張高畫質大頭貼 (Data URI Base64 編碼)，很可能會瞬間把容量塞爆導致整站崩潰。

解法：利用 HTML5 <canvas> 在前端瞬間無損壓縮！

這是一個超級專業的黑科技，我們可以在拿到使用者圖片的瞬間，在背景畫出一張 200x200 像素的隱形畫布，把大圖縮進去，再把它輸出。這會讓一張好幾 MB 的圖片瞬間縮小到只有大約 10KB，完美寫入 localStorage！

```
// 零後端：大頭貼自動壓縮器 (HTML5 Canvas 技巧)
const handleAvatarChange = (e) => {
  const file = e.target.files?.[0];
  if (!file) return;

  const reader = new FileReader();
  reader.onload = (event) => {
    const img = new Image();
    img.onload = () => {
      // 1. 建立隱形畫布
      const canvas = document.createElement('canvas');
      const MAX_SIZE = 200; // 最寬或最高 200px

      let { width, height } = img;
      if (width > height) {
        if (width > MAX_SIZE) { height *= MAX_SIZE / width; width = MAX_SIZE; }
      } else {
        if (height > MAX_SIZE) { width *= MAX_SIZE / height; height = MAX_SIZE; }
      }

      // 2. 寫入畫布並等比例縮放
      canvas.width = width;
      canvas.height = height;
      const ctx = canvas.getContext('2d');
      ctx?.drawImage(img, 0, 0, width, height);

      // 3. 輸出成 Base64 字串，大功告成！直接可以存入 localStorage
      const tinyBase64Data = canvas.toDataURL('image/jpeg', 0.8);
      console.log("壓縮完成：", tinyBase64Data);
    };
    img.src = event.target?.result; // 觸發 img.onload
  };
  reader.readAsDataURL(file); // 讀取原始大檔
};
```

📱 終極進化：讓網頁變身超級手機 App (PWA)

這套系統不只在電腦的超薄邊框螢幕上很炫酷，它現在還是一台貨真價實的「跨平台應用程式」！我們使用了現代化的 **漸進式網頁應用程式 (PWA)** 架構，讓它擁有能夠像一般 App 一樣「掃描 QR Code 安裝到手機桌面」的神奇能力。

1. 手機專屬「無縫底部選單」架構

在傳統電腦版，我們設計了極具質感的「隱藏式側邊欄 (w-64)」；但是在手機上，這會完全吃光寶貴的空間。解法非常簡單直接：我們讓大螢幕版 (sm:flex) 保有左側面板；但在手機極限畫面上，我們實作了一條純黑玻璃質感的 **底部控制列 (Bottom Tab Bar)**，完全隱藏在 sm:hidden fixed bottom-0 屬性裡。只要用手機瀏覽，導航列會自動從左邊移到底部，擁有高階 iOS App 的沉浸視角。

2. QR Code 一鍵轉生 APP

當你在電腦瀏覽器中點擊右上角的「Share」圖標，會彈出我們精心手刻的 QR Code 視窗。這裡整合了強大的 qrcode.react 與 vite-plugin-pwa 插件：

- vite.config.ts 核心設定**：我們宣告了這個專案的 manifest，綁定了專案名稱如 QuantTrd、背景為深邃黑 (#0a0a0f)，並且讓它 display: 'standalone' —— 這代表當專案被安裝到手機時，最上方的醜陋瀏覽器網址列會被強制關閉消失！
- 網路共享**：我們打開了 Vite 的 host: true。這讓你只要跟電腦連著同一個 WiFi，拿出手機鏡頭一掃 QR 視窗，這套 QuantTrd 金融分析系統就會突破時空，直接跑到你的手掌心。

恭喜你！跟著這些技術，你已經將一份單純的切版作品，升級成整合「實質偏好記憶、無盡新聞瀑布流、跨螢幕無痛轉移、並支援本機影像壓縮演算」的 **生產力級別高階 Web APP 完全體**！

🚀 完美落地：Vercel 佈署與伺服器代理 (Proxy)

當你準備把作品推向全世界時，我們選擇了當今最強大的前端雲端平台 **Vercel** 來進行上架。但在這之中，有一個初學者必定會踩到的大坑：**CORS 跨域請求阻擋與 API Proxy 斷線問題**！

1. 為何本地會通，上線卻壞了？

我們在開發時，利用了 vite.config.ts 中的 proxy 將我們對 /api/finance 的請求，偷偷轉發給了 Yahoo 的股票主機。這在自己電腦上非常管用。但注意！當你將程式碼 Push 到 GitHub 並交給 Vercel 時，Vercel 只會把編譯完的靜態 HTML/JS 檔拿去渲染，它完全不知道你的 Vite 設定有這條代理規則！

2. 解法：撰寫 vercel.json 伺服器代理配置

為了解決這個問題，我們必須在專案的根目錄，直接命令 Vercel 的伺服器引擎去幫我們抓資料。我們新增了 vercel.json 檔案：

```
{
  "rewrites": [
    {
      "source": "/api/finance/:match*",
      "destination": "https://query1.finance.yahoo.com/:match*"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

這個「黑魔術」會讓 Vercel 建立一個無伺服器 (Serverless) 路由。任何打向 /api/finance 的 API，Vercel 雲端主機都會充當白手套，替你向金融伺服器獲取資料，達成完美繞過所有 CORS 阻擋的神級技巧！

3. 客製化真實新聞分頁系統 (Pagination)

為了解決第三方元件 (如 TradingView) 的免費數量限制，我們在 Views.tsx 自己手刻了新聞處理器。我們把抓下來的龐大新聞陣列 (Array)，利用狀態機 useState 搭配超簡單的切片數學：

```
const currentNews = news.slice((page - 1) * pageSize, page * pageSize);
```

我們因此成功做出了擁有 1, 2, 3, 4, 5 實體翻頁按鈕的超炫新聞牆。現在，你的系統不僅 UI 完美，連背後的資料處理邏輯也達到了業界級別的扎實度！